Reasonable Quaternion Arithmetic for `polyForth` on Volatco

Representation, Implementation, and Performance Evidence

Cartheur

June 8, 2026

Abstract

This paper studies whether quaternion arithmetic is a reasonable computational choice for `polyForth` on Volatco, an asynchronous F18A-based platform intended for low-power control, robotics, and experimental machine-intelligence workloads. The standard of reasonableness adopted here is deliberately narrow: the representation must be simple to encode in a stack language, compact enough to respect the constraints of an 18-bit node, and measurably competitive with plausible alternatives on representative rotational workloads. The repository therefore combines source code, benchmark definitions, and manuscript material so that the argument can be revised against hardware measurements rather than asserted from algebra alone.
TODO: Revise the abstract once real Volatco measurements exist so that it states actual findings rather than the present research intent.

1 Introduction

Quaternion methods are attractive in rotational computation because they offer a compact composition law and avoid some of the representational overhead of matrix-based formulations. Those advantages are familiar in conventional software environments, but they cannot simply be assumed on a tiny asynchronous multicomputer running `polyForth`. On such a platform, the right question is not whether quaternions are elegant, but whether they are operationally reasonable.

Volatco is a useful target for this question because it is explicitly Forth-native and designed around deterministic, low-latency execution. Its published documentation emphasizes reproducible `polyForth` experiments with explicit pass/fail criteria, power measurement points, and bounded-latency behavior [1, 2]. That combination makes it possible to write a paper that is both mathematical and empirical.
TODO: Add citations from the local quaternion corpus for the mathematical motivation, especially references on quaternion rotation and computational advantages or limitations relative to matrices.

2 Claim

The working claim of this paper is modest:

> A fixed-point quaternion representation is a reasonable rotational algebra for `polyForth` on Volatco when it yields compact code, controlled state, and reproducible performance under explicitly measured workloads.

The claim is intentionally narrower than "quaternions are best." It is a claim about suitability under a specific architecture, runtime style, and evidence standard.

TODO: Replace the current claim text with a version tied to measured thresholds, for example code size, elapsed time, or power bounds, once those numbers exist.

3 Representations Under Comparison

The implementation scaffold in this repository uses two competing representations. The quaternion kernel stores (a, b, c, d), where a is the scalar part and b, c, d are the vector components. The comparison kernel stores a 3×3 rotation matrix in row-major order plus a three-component vector. This side-by-side structure is important: the paper should not ask whether quaternions are elegant in the abstract, but whether they remain reasonable once compared to a direct matrix approach under the same tooling and target constraints.

The quaternion layout maps directly to Hamilton's multiplication rule. The current draft keeps the representation abstract enough to allow either integer arithmetic or scaled fixed-point arithmetic after target measurements determine which is more practical on Volatco.

TODO: Choose the actual numeric model for the paper: plain integer, fixed-point with explicit scale factor, or a mixed representation. Document the overflow and precision tradeoffs for the chosen model.

The scalar-first ordering is not mathematically necessary. It is adopted here for engineering clarity, because conjugation, norm-squared, and scalar-vector decomposition can be written with simple address-based stack effects in Forth.

TODO: Add one short worked example showing the in-memory layout of a quaternion and a matrix on the target Forth system.

4 Implementation Strategy

The initial Forth implementation favors transparency over optimization. Core quaternion words are provided for addition, subtraction, conjugation, norm-squared, Hamilton product, and vector rotation via qvq^{-1}, implemented as `q * v * conjugate(q)`. A companion matrix kernel performs matrix-vector and matrix-matrix multiplication in the same address-based style. This is the correct place to start because a paper cannot defend an optimized kernel unless it first presents readable reference implementations for both the proposed and comparison methods.

TODO: Add an appendix excerpt of the actual reference Forth code and note any edits required for the exact Volatco `polyForth` image.

After reference behavior is established, optimization can proceed in three directions:

1. reduce stack traffic
2. remove scratch storage where the target system permits
3. distribute work across nodes when the communication cost is justified

TODO: State whether this paper evaluates only single-node kernels or also includes multi-node decomposition across the Volatco mesh.

The deployment method also matters to reproducibility. In the current repo, the Volatco-facing source is treated as block-resident `polyForth` source rather than as an ephemeral terminal transcript. A generated `VOLATCO` project image reserves blocks 0 through 9, uses block 10 as the application load block, and places the quaternion source in blocks 11 through 30. The intended operator action is therefore to enter `AFORTH` and issue `10 LOAD`. This detail is important because

it makes the experiment reproducible in the same block-oriented style used by the surrounding saneForth workflow.

5 Measurement Plan

The benchmark argument in this repository compares quaternion operations against a direct 3×3 rotation matrix kernel written in the same style. That comparison is necessary because the matrix method spends more storage but may win in some workloads by avoiding quaternion conjugation or normalization overhead.

Each benchmark should record:

- iteration count
- elapsed time
- code footprint
- current or power measurement
- watchdog stability during long runs

The results should be treated as evidence about suitability for Volatco, not as universal claims about all Forth systems.

The first hardware experiment should not yet be a timing experiment. It should be a deployment and correctness experiment that validates the block-image path itself. In practical terms, that means loading the generated project image, entering `AFORTH`, issuing `10 LOAD`, and checking that both the quaternion and matrix report words produce the same rotated vector $(10, -20, -30)$. This is not the paper's central result, but it is a necessary methodological step. Without it, any later performance numbers would rest on an uncertain loading procedure and could reasonably be challenged as artifact-dependent.

A conventional reference platform may still be useful, but only as a secondary calibration experiment. If included, it should use the repository's `gforth` reference implementation on a machine such as a Raspberry Pi and should be presented as appendix evidence about the qualitative quaternion-versus-matrix tradeoff, not as the primary basis of the paper's claim.

TODO: Insert a real results table here with at least quaternion rotation versus matrix-vector rotation, including iteration count, elapsed time, and power measurement point.

TODO: Document the timing method precisely: serial timestamping, on-board counter, external logic capture, or another method.

6 Expected Contribution

If the measurements support the working claim, the paper contributes a concrete case study in how a classical algebraic formalism can be adapted to a modern asynchronous Forth-native platform. If the measurements do not support the claim, that result is still valuable because it clarifies where quaternion methods cease to be worth their conceptual appeal under tight embedded constraints.

The comparison with a matrix kernel is especially important here. A paper that only demonstrates that quaternion code runs on Volatco would be incomplete. The interesting result is whether quaternion state, composition, and rotation support a better tradeoff than a simpler matrix pipeline once timing, code size, and power are measured on the same board.

TODO: Once data exists, make this section argue from the observed tradeoff rather than from expected contribution alone.

7 Current Status

At present, the repository contains:

- an address-based quaternion kernel for Forth
- a companion 3×3 matrix kernel
- a benchmark scaffold for repeated arithmetic and rotation workloads
- a reproducibility protocol for eventual Volatco measurements

It does not yet contain:

- confirmed target-specific `polyForth` adaptations
- hardware measurements from a real Volatco board
- a completed citation apparatus grounded in the local quaternion corpus

The present engineering assumptions about Volatco derive from the public site and documentation rather than from private hardware notes [1, 2].
TODO: Replace this status section with a brief methodology section after the first hardware campaign is complete.

8 Next Revision Points

- replace placeholder prose with citations from the local quaternion corpus
- confirm `polyForth` compatibility on Volatco
- choose and document a fixed-point scaling rule
- add measured benchmark results
- interpret the quaternion results against the matrix baseline
- narrow the claim to match the data

9 Focused TODO List

- Gather mathematical citations from the local quaternion papers and books.
- Confirm the exact Volatco `polyForth` dialect and any source edits needed for the current Forth files.
- Choose the numeric representation and document scale, range, and overflow behavior.
- Run quaternion and matrix benchmarks on real Volatco hardware.
- Add tables for timing, power, and code size.
- Rewrite the abstract, claim, and discussion around measured evidence.

References

[1] Volatco. `https://volatco.tech/`. Accessed 2026-06-08.

[2] Volatco documentation. `https://volatco.github.io/`. Accessed 2026-06-08.

A Benchmark Protocol Summary

The benchmark protocol should report, at minimum, board revision, runtime revision, exact source, iteration count, elapsed time, power measurement point, code size, pass/fail outcome, and notes on watchdog or reset behavior. The baseline workload set should include repeated quaternion addition, Hamilton product, norm-squared, vector rotation, matrix-vector rotation, and matrix-matrix multiplication.